```
# 03_real_figures.py
# Makes REAL image panels from the dataset to replace the synthetic illustrations in the deck.
# Run after 02. No GPU needed. Outputs go to  <ROOT>/figs/real/  (download the folder).
```

```
# %% [1] Imports and settings
import os, json
import numpy as np
import cv2
import tifffile
import matplotlib
matplotlib.use("Agg")
import matplotlib.cm as cm

ROOT = os.environ.get("AMAZON_ROOT", "/content/drive/MyDrive/amazon_project")
try:
    from google.colab import drive
    drive.mount("/content/drive")
except ImportError:
    pass

CACHE, OUT = f"{ROOT}/cache", f"{ROOT}/figs/real"
os.makedirs(OUT, exist_ok=True)
paths = json.load(open(f"{ROOT}/paths.json"))
LABELS = json.load(open(f"{CACHE}/labels.json"))
names = json.load(open(f"{CACHE}/names.json"))
Y = np.load(f"{CACHE}/Y.npy")
O = json.load(open(f"{CACHE}/order.json"))
BS = json.load(open(f"{CACHE}/band_stats.json"))
lo, hi = np.array(BS["lo"]), np.array(BS["hi"])
rng = np.random.RandomState(7)


def read_tif(n):
    a = tifffile.imread(os.path.join(paths["tif_dir"], n + ".tif")).astype(np.float32)
    if a.ndim == 3 and a.shape[0] == 4 and a.shape[-1] != 4:
        a = np.transpose(a, (1, 2, 0))
    return a


def stretch(a, band):
    return np.clip((a[..., band] - lo[band]) / (hi[band] - lo[band] + 1e-6), 0, 1)


def rgb(a, gamma=0.8):
    img = np.stack([stretch(a, O["r"]), stretch(a, O["g"]), stretch(a, O["b"])], -1)
    return (255 * img ** gamma).astype(np.uint8)


def save(img, name, size=512):
    img = cv2.resize(img, (size, size), interpolation=cv2.INTER_CUBIC)
    cv2.imwrite(f"{OUT}/{name}.png", cv2.cvtColor(img, cv2.COLOR_RGB2BGR))


def pick(label, max_tags=None):
    j = LABELS.index(label)
    cand = np.where(Y[:, j] == 1)[0]
    cand = cand[np.argsort(Y[cand].sum(1))]          # prefer tiles with few other labels
    if max_tags:
        cand = cand[Y[cand].sum(1) <= max_tags]
    return int(rng.choice(cand[: max(10, len(cand) // 10)]))
```

```
# %% [2] One real example per label (replaces the chip gallery)
show = {}
for lab in ["primary", "agriculture", "road", "water", "partly_cloudy", "haze",
            "slash_burn", "artisinal_mine", "selective_logging", "cloudy"]:
    if lab not in LABELS:
```

```
        continue
    i = pick(lab, 3 if lab in ("primary", "water", "haze", "partly_cloudy") else None)
    show[lab] = i
    save(rgb(read_tif(names[i])), f"real_chip_{lab}")
    print(lab, names[i], [LABELS[j] for j in np.where(Y[i] == 1)[0]])
```

 # %% [3] RGB / false-colour NIR / NDVI trio (replaces the NDVI slide images)

```
want = [LABELS.index(l) for l in ["agriculture", "road", "primary"]]
cand = np.where(Y[:, want].all(1))[0]
i = int(rng.choice(cand))
a = read_tif(names[i])
save(rgb(a), "real_rgb")
cir = np.stack([stretch(a, O["nir"]), stretch(a, O["r"]), stretch(a, O["g"])], -1)
save((255 * cir ** 0.8).astype(np.uint8), "real_cir")
ndvi = (a[..., O["nir"]] - a[..., O["r"]]) / (a[..., O["nir"]] + a[..., O["r"]] + 1e-6)
col = cm.RdYlGn((np.clip(ndvi, -0.2, 0.9) + 0.2) / 1.1)[..., :3]
save((255 * col).astype(np.uint8), "real_ndvi")
print("trio tile:", names[i], [LABELS[j] for j in np.where(Y[i] == 1)[0]])
```

 # %% [4] Augmentation examples (replaces the augmentation slide images)

```
base = rgb(read_tif(names[show.get("agriculture", i)]))
variants = [base, np.rot90(base).copy(), base[:, ::-1].copy(),
            np.clip(base[::-1].astype(np.float32) * 1.3, 0, 255).astype(np.uint8)]
for k, v in enumerate(variants):
    save(v, f"real_aug_{k}")
```

 # %% [5] Multi-label example tile with 4+ tags

```
cand = np.where(Y.sum(1) >= 5)[0]
j = int(rng.choice(cand)) if len(cand) else i
save(rgb(read_tif(names[j])), "real_multilabel")
print("multi-label tile tags:", [LABELS[k] for k in np.where(Y[j] == 1)[0]])
```

 # %% [6] Mosaic for the title and closing slides (1600 x 900)

```
S = 200
prim = np.where(Y[:, LABELS.index("primary")] == 1)[0]
agri = np.where(Y[:, LABELS.index("agriculture")] == 1)[0]
tiles = []
for _ in range(8 * 5):
    pool = agri if rng.rand() < 0.45 else prim
    tiles.append(cv2.resize(rgb(read_tif(names[int(rng.choice(pool))])), (S, S)))
rows = [np.concatenate(tiles[r * 8:(r + 1) * 8], 1) for r in range(5)]
mosaic = np.concatenate(rows, 0)[:900]
cv2.imwrite(f"{OUT}/real_mosaic.png", cv2.cvtColor(mosaic, cv2.COLOR_RGB2BGR))
print("Saved to", OUT, "- download this folder and swap the images into the deck")
```